Chapter 14. Role-Based Access Control for Kubernetes

At this point, nearly every Kubernetes cluster you encounter has role-based access control (RBAC) enabled. So you have likely encountered RBAC before. Perhaps you initially couldn't access your cluster until you used some magical command to add a RoleBinding to map a user to a role. Even though you may have had some exposure to RBAC, you may not have had a great deal of experience understanding RBAC in Kubernetes, including what it is for and how to use it.

Role-based access control provides a mechanism for restricting both access to and actions on Kubernetes APIs to ensure that only authorized users have access. RBAC is a critical component to both harden access to the Kubernetes cluster where you are deploying your application and (possibly more importantly) prevent unexpected accidents where one person in the wrong namespace mistakenly takes down production when they think they are destroying their test cluster.

NOTE

While RBAC can be quite useful in limiting access to the Kubernetes API, it's important to remember that anyone who can run arbitrary code inside the Kubernetes cluster can effectively obtain root privileges on the entire cluster. There are approaches that you can take to make such attacks harder and more expensive, and a correct RBAC setup is part of this defense. But if you are focused on hostile multitenant security, RBAC by itself is sufficient to protect you. You must isolate the Pods running in your cluster to provide effective multitenant security. Generally this is done using hypervisor isolated containers or a container sandbox.

Before we dive into the details of RBAC in Kubernetes, it's valuable to have a high-level understanding of RBAC as a concept, as well as authentication and authorization more generally.

Every request to Kubernetes is first *authenticated*. Authentication provides the identity of the caller issuing the request. It could be as simple as saying that the request is unauthenticated, or it could integrate deeply with a pluggable authentication provider (e.g.,

Azure Active Directory) to establish an identity within that third-party system. Interestingly enough, Kubernetes does not have a built-in identity store, focusing instead on integrating other identity sources within itself.

Once users have been authenticated, the authorization phase determines whether they are authorized to perform the request. Authorization is a combination of the identity of the user, the resource (effectively the HTTP path), and the verb or action the user is attempting to perform. If the particular user is authorized to perform that action on that resource, then the request is allowed to proceed. Otherwise, an HTTP 403 error is returned. Let's dive into this process.

Role-Based Access Control

To properly manage access in Kubernetes, it's critical to understand how identity, roles, and role bindings interact to control who can do what with which resources. At first, RBAC can seem like a challenge to understand, with a series of interconnected, abstract concepts; but once it's understood, you can be confident in your ability to manage cluster access.

Identity in Kubernetes

Every request to Kubernetes is associated with some identity. Even a request with no identity is associated with the `system:unauthenticated` group. Kubernetes makes a distinction between user identities and service account identities. Service accounts are created and managed by Kubernetes itself and are generally associated with components running inside the cluster. User accounts are all other accounts associated with actual users of the cluster, and often include automation like continuous delivery services that run outside the cluster.

Kubernetes uses a generic interface for authentication providers. Each of the providers supplies a username and, optionally, the set of groups to which the user belongs. Kubernetes supports a number of authentication providers, including:

- HTTP Basic Authentication (largely deprecated)
- x509 client certificates
- Static token files on the host
- Cloud authentication providers, such as Azure Active Directory and AWS Identity and Access Management (IAM)
- Authentication webhooks

While most managed Kubernetes installations configure authentication for you, if you are deploying your own authentication, you will need to configure flags on the Kubernetes API server appropriately.

You should always use different identities for different applications in your cluster. For example, you should have one identity for your production frontends, a different identity for the production backends, and all production identities should be distinct from development identities. You should also have different identities for different clusters. All of these identities should be machine identities that are not shared with users. You can either use Kubernetes Service Accounts for achieving this, or you can use a Pod identity provider supplied by your identity system; for example, Azure Active Directory supplies an open source identity provider for Pods as do other popular identity providers.

Understanding Roles and Role Bindings

Identity is just the beginning of authorization in Kubernetes. Once Kubernetes knows the identity of the request, it needs to determine if the request is authorized for that user. To achieve this, it uses roles and role bindings.

A *role* is a set of abstract capabilities. For example, the `appdev` role might represent the ability to create Pods and Services. A *role binding* is an assignment of a role to one or more identities. Thus, binding the `appdev` role to the user identity `alice` indicates that Alice has the ability to create Pods and Services.

Roles and Role Bindings in Kubernetes

In Kubernetes, two pairs of related resources represent roles and role bindings. One pair is scoped to a namespace (Role and RoleBinding), while the other pair is scoped to the cluster (ClusterRole and ClusterRoleBinding).

Let's examine Role and RoleBinding first. Role resources are namespaced and represent capabilities within that single namespace. You cannot use namespaced roles for non-namespaced resources (e.g., CustomResourceDefinitions), and binding a RoleBinding to a role only provides authorization within the Kubernetes namespace that contains both the Role and the RoleBinding.

As a concrete example, here is a simple role that gives an identity the ability to create and modify Pods and Services:

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: default
  name: pod-and-services
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
```

To bind this Role to the user `alice`, we need to create a RoleBinding that looks as follows. This role binding also binds the group `mydevs` to the same role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: default
  name: pods-and-services
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: alice
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: mydevs
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-and-services
```

Sometimes you need to create a role that applies to the entire cluster, or you want to limit access to cluster-level resources. To achieve this, you use the ClusterRole and ClusterRoleBinding resources. They are largely identical to their namespaced peers, but are cluster-scoped.

Verbs for Kubernetes roles

Roles are defined in terms of both a resource (e.g., Pods) and a verb that describes an action that can be performed on that resource. The verbs correspond roughly to HTTP methods. The commonly used verbs in Kubernetes RBAC are listed in Table 14-1.

Table 14-1. Common Kubernetes RBAC verbs

Verb	HTTP method	Description
`create`	`POST`	Create a new resource.
delete	DELETE	Delete an existing resource.
`get`	`GET`	Get a resource.
list	GET	List a collection of resources.
`patch`	`PATCH`	Modify an existing resource via a partial change.
update	PUT	Modify an existing resource via a complete object.
`watch`	`GET`	Watch for streaming updates to a resource.
proxy	GET	Connect to resource via a streaming WebSocket proxy.

Using built-in roles

Designing your own roles can be complicated and time-consuming. Kubernetes has a large number of built-in cluster roles for well-known system identities (e.g., a scheduler) that require a known set of capabilities. You can view these by running:

```
$ kubectl get clusterroles
```

While most of these built-in roles are for system utilities, four are designed for generic end users:

- The `cluster-admin` role provides complete access to the entire cluster.
- The `admin` role provides complete access to a complete namespace.
- The `edit` role allows an end user to modify resources in a namespace.
- The `view` role allows for read-only access to a namespace.

Most clusters already have numerous ClusterRole bindings set up, and you can view these bindings with `kubectl get clusterrolebindings`.

Auto-reconciliation of built-in roles

When the Kubernetes API server starts up, it automatically installs a number of default ClusterRoles that are defined in the code of the API server itself. This means that if you modify any built-in cluster role, those modifications are transient. Whenever the API server is restarted (e.g., for an upgrade), your changes will be overwritten.

To prevent this from happening, before you make any other modifications, you need to add the `rbac.authorization.kubernetes.io/autoupdate` annotation with a value of `false` to the built-in ClusterRole resource. If this annotation is set to `false`, the API server will not overwrite the modified ClusterRole resource.

WARNING

By default, the Kubernetes API server installs a cluster role that allows `system:unauthenticated` users access to the API server's API discovery endpoint. For any cluster exposed to a hostile environment (e.g., the public internet) this is a bad idea, and there has been at least one serious security vulnerability via this exposure. If you are running a Kubernetes service on the public internet or an other hostile environment, you should ensure that the `--anonymous-auth=false` flag is set on your API server.

Techniques for Managing RBAC

Managing RBAC for a cluster can be complicated and frustrating. Possibly more concerning is that misconfigured RBAC can lead to security issues. Fortunately, there are several tools and techniques that make managing RBAC easier.

Testing Authorization with can-i

The first useful tool is the `auth can-i` command for `kubectl`. This tool is used for testing whether a specific user can perform a specific action. You can use `can-i` to validate configuration settings as you configure your cluster, or you can ask users to use the tool to validate their access when filing errors or bug reports.

In its simplest usage, the `can-i` command takes a verb and a resource. For example, this command will indicate if the current `kubectl` user is authorized to create Pods:

```
$ kubectl auth can-i create pods
```

You can also test subresources like logs or port-forwarding with the `--subresource` command-line flag:

```
$ kubectl auth can-i get pods --subresource=logs
```

Managing RBAC in Source Control

Like all resources in Kubernetes, RBAC resources are modeled using YAML. Given this text-based representation, it makes sense to store these resources in version control, which allows for accountability, auditability, and rollback.

The `kubectl` command-line tool provides a `reconcile` command that operates somewhat like `kubectl apply` and will reconcile a set of roles and role bindings with the current state of the cluster. You can run:

```
$ kubectl auth reconcile -f some-rbac-config.yaml
```

If you want to see changes before they are made, you can add the `--dry-run` flag to the command to output, but not apply, the changes.

Advanced Topics

Once you orient to the basics of role-based access control, it is relatively easy to manage access to a Kubernetes cluster. But when managing a large number of users or roles, there are additional advanced capabilities you can use to manage RBAC at scale.

Aggregating ClusterRoles

Sometimes you want to be able to define roles that are combinations of other roles. One option would be to simply clone all of the rules from one ClusterRole into another ClusterRole, but this is complicated and error-prone, since changes to one ClusterRole aren't automatically reflected in the other. Instead, Kubernetes RBAC supports the usage of an *aggregation rule* to combine multiple roles in a new role. This new role combines all of the capabilities of all of the aggregate roles, and any changes to any of the constituent subroles will automatically be propogated back into the aggregate role.

As with other aggregations or groupings in Kubernetes, the ClusterRoles to be aggregated are specified using label selectors. In this particular case, the `aggregationRule` field in the ClusterRole resource contains a `clusterRoleSelector` field, which in turn is a label selector. All ClusterRole resources that match this selector are dynamically aggregated into the `rules` array in the aggregate ClusterRole resource.

A best practice for managing ClusterRole resources is to create a number of fine-grained cluster roles and then aggregate them to form higher-level or broader cluster roles. This is how the built-in cluster roles are defined. For example, you can see that the built-in `edit` role looks like this:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: edit
  ...
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
      rbac.authorization.k8s.io/aggregate-to-edit: "true"
...
```

This means that the `edit` role is defined to be the aggregate of all ClusterRole objects that have a label of `rbac.authorization.k8s.io/aggregate-to-edit` set to `true`.

Using Groups for Bindings

When managing a large number of people in different organizations with similar access to the cluster, it's generally a best practice to use groups to manage the roles that define access, rather than individually adding bindings to specific identities. When you bind a group to a Role or ClusterRole, anyone who is a member of that group gains access to the resources and verbs defined by that role. Thus, to enable any individual to gain access to the group's role, that individual needs to be added to the group.

Using groups is a preferred strategy for managing access at scale for several reasons. The first is that in any large organization, access to the cluster is defined in terms of the team that someone is part of, rather than their specific identity. For example, someone who is part of the frontend operations team will need access to both view and edit the resources associated with the frontends, while they may only need view/read access to resources associated with the backend. Granting privileges to a group makes the associa-

tion between the specific team and its capabilities clear. When granting roles to individuals, it's much harder to clearly understand the appropriate (i.e., minimal) privileges required for each team, especially when an individual may be part of multiple teams.

Additional benefits of binding roles to groups instead of individuals are simplicity and consistency. When someone joins or leaves a team, it is straightforward to simply add or remove them to or from a group in a single operation. If you instead have to remove a number of different role bindings for their identity, you may either remove too few or too many bindings, resulting in unnecessary access or preventing them from being able to do necessary actions. Additionally, because there is only a single set of group role bindings to maintain, you don't have to do lots of work to ensure that all team members have the same, consistent set of permissions.

NOTE

Many cloud providers support integrations onto their identity and access management platforms so that users and groups from those platforms can be used in conjunction with Kubernetes RBAC.

Many group systems enable "just in time" (JIT) access, such that people are only temporarily added to a group in response to an event (say, a page in the middle of the night) rather than having standing access. This means that you can both audit who had access at any particular time and ensure that, in general, even a compromised identity can't have access to your production infrastructure.

Finally, in many cases, these same groups are used to manage access to other resources, from facilities to documents and machine logins. Thus, using the same groups for access control to Kubernetes dramatically simplifies management.

To bind a group to a ClusterRole, use a Group kind for the `subject` in the binding:

```
...
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: my-great-groups-name
...
```

In Kubernetes, groups are supplied by authentication providers. There is no strong notion of a group within Kubernetes, only that an identity can be part of one or more

groups, and those groups can be associated with a Role or ClusterRole via a binding.

Summary

When you begin with a small cluster and a small team, it is sufficient to have every member of the team have equivalent access to the cluster. But as teams grow and products become more mission critical, limiting access to parts of the cluster is crucial. In a well-designed cluster, access is limited to the minimal set of people and capabilities needed to efficiently manage the applications in the cluster.

Understanding how Kubernetes implements RBAC and how those capabilities can be used to control access to your cluster is important for both developers and cluster administrators. As with building out testing infrastructure, best practice is to set up RBAC earlier rather than later. It's far easier to start with the right foundation than to try to retrofit it later on. Hopefully, the information in this chapter has provided the necessary grounding for adding RBAC to your cluster.

Previous chapter
< 13. ConfigMaps and Secrets

Next chapter
15. Service Meshes >